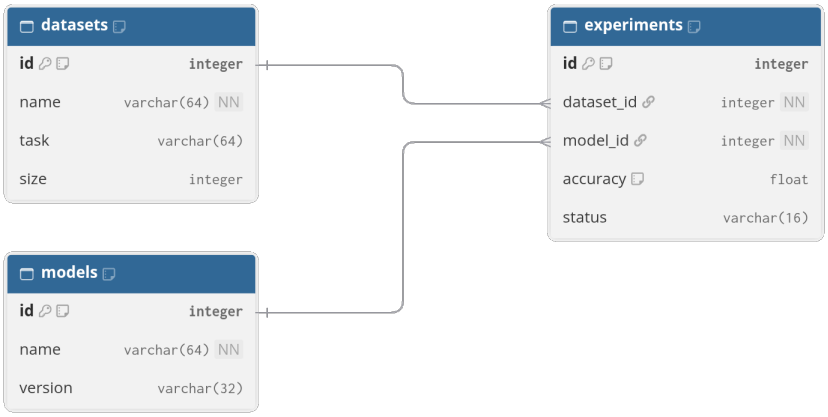
MiniDB · лабораторная работа №1

Схема, устройство и формат хранения

Мини-СУБД реализована на Rust без внешних Rust-зависимостей и готовых СУБД. JSON не используется как основное хранилище. SQL-консоль создаёт, ищет, изменяет и удаляет строки; данные сохраняются после перезапуска.

Три связанные таблицы



datasets(id, name, task, size); models(id, name, version); experiments(id, dataset_id, model_id, accuracy, status). PK — id. FK experiments ссылаются на PK datasets/models. Проверяются уникальность, авто-ID, наличие родителя и RESTRICT.

Собственные файлы

На таблицу: имя_schema — CREATE TABLE, VERSION, GENERATION и счётчик auto-ID; имя_N — единственные текущие данные, N берётся из GENERATION. UTF-8: заголовок, состояния LIVE/DEAD/FREE, значения через Tab. Строки заключены в кавычки, специальные символы экранированы. INT=i64, FLOAT=f64, STRING(n)=UTF-8 до n байт.

ALTER (ADD/DROP COLUMN, снятие FK) и VACUUM собирают живые строки в новом поколении: sync данных → атомарная замена схемы → sync каталога → удаление прежнего файла. Открытие убирает осиротевшие поколения. ALTER увеличивает VERSION и GENERATION; VACUUM — только GENERATION, если есть мёртвые слоты. Максимум auto-ID сохраняется.

Архитектура

sql.rs → Database (PK/FK, запросы, backup) → Table → TextFile. Описатель разделён на Schema (поля и ограничения) и Layout (NULL-bitmap, ширины, смещения в RAM). Row заимствует оба описателя и байты слота. UTF-8 декодируется в RAM; обычный CRUD сохраняет атомарный снимок текущего файла. Индекс PK → offset адресует слот в RAM, не байт текстового файла.

ADD/DROP перекодируют живые строки одной таблицы: оставшиеся поля сопоставляются по имени, новые получают NULL. Ядро не пересобирается; одновременно в RAM находятся старый и новый буферы. До переключения на диске тоже лежат два поколения.

MiniDB · лабораторная работа №1

Запуск, примеры операций и результат индекса

Локально нужен Rust/Cargo 1.89+. Для Docker Rust на хосте не нужен: один раз выполните docker compose build. Два режима запуска:

./run.sh	пустая база
./run.sh sample	база из ТЗ
docker compose run --rm minidb	пустая база
docker compose run --rm minidb sample	база из ТЗ

Локальные пути: db/main и db/sample; в Docker: /data/db и /data/sample. sample — текстовый SQL-файл наполнения. Повторный запуск открывает сохранённую базу. Пример: 5 datasets, 5 models и 20 experiments; первые три experiments — из ТЗ. HELP; показывает справку.

Операции и ожидаемый результат

```
CREATE TABLE notes (id INT PRIMARY KEY, text STRING(64), score FLOAT);
INSERT INTO datasets (name, task, size) VALUES ('demo', 'classification', 43);
SELECT id, name FROM datasets WHERE size >= 40;
UPDATE datasets SET size = 44 WHERE name = 'demo';
DELETE FROM datasets WHERE name = 'demo';
INSERT INTO experiments VALUES (NULL, 999, 1, 0.9, 'running');
EXIT;
```

INSERT выдаёт авто-ID; UPDATE и DELETE меняют по одной строке. INSERT с dataset 999 отклоняется из-за FK. Повторите запуск с sample, затем SHOW TABLES; SELECT * FROM experiments; — notes и 20 experiments сохранились. Поддерживаются шесть сравнений (=, !=, >, <, >=, <=), AND/OR, выбор полей и INNER JOIN.

Сравнение поиска с индексом и без

Release: 5000 строк, 1000 одинаковых запросов. Наполнение, открытие и sync вне таймера. Данные уже в RAM; это не измерение дискового I/O. Контрольные суммы всех трёх способов совпали: 2486500.

Просмотр — O(числа слотов); HashMap по равенству INT PK — в среднем O(1), затем проверяется весь WHERE. Диапазоны и другие поля требуют просмотра. Индекс строится при открытии и обновляется при CRUD; после переписи перестраивается.

Полный просмотр	66.159 мс
Индекс get(id)	0.143 мс
SELECT WHERE id = …	0.134 мс

Повторить: cargo run --locked --offline --release --example bench -- scratch/bench 5000. Это один локальный замер; результат зависит от машины.

Исходники: https://github.com/AI-paca/MiniDB. Данные примера: db/sample; схема dbdiagram.io: docs/schema.dbml; результаты замера: docs/benchmark.csv.